From a 2010 Prototype to Contraction Hierarchies in Three Languages: Engineering k Shortest Paths with Eppstein, Yen–Lawler, Arc Flags and PHAST

An experiment in algorithm engineering with Claude

Nick Dinges*

November 2026

Abstract

We report on turning a small C# implementation of Eppstein's k shortest paths algorithm, written in 2010, into a library that combines several modern techniques: Eppstein's enumeration with lazily built persistent sidetrack heaps, a detour corridor that bounds both the shortest-path tree and the enumeration, Yen's algorithm with Lawler's improvement and A* spur searches for loopless paths, arc flags with external graph partitions, and a contraction hierarchy whose PHAST sweep computes the full shortest-path tree. The library exists in three languages with the same interface and results: C# and Go, which are garbage collected, and Rust, which is not. All three share golden test data and an identical benchmark, so their timings can be compared on exactly the same graphs. On grid graphs with up to one million vertices, all three implementations are within a factor of 1.3 of each other for the shortest-path tree. Rust is the fastest in most scenarios, by 10–40%. The largest gaps (up to 4× on small inputs) come from .NET's tiered JIT compilation, not from garbage collection: disabling Go's collector or making .NET's rare has no measurable effect, because all three use the same flat, allocation-free data layout. The algorithmic choices dominate by far: the original 2010 implementation needs 40.3 s for the first path on a 12×12 acyclic grid where the current one needs 0.12 ms, and a corridor turns a 69 ms query into 0.5 ms. Finally, we describe how the work was done: the whole transformation, including this paper, was generated with Claude in about two days, directed by the author and based on the author's old code and on ideas from a domain expert. We report what worked, what the AI got wrong, and how tests caught it.

1 Introduction

The k shortest paths problem asks for the k shortest paths between two vertices s and t of a weighted directed graph, in order of weight. It appears in route planning (alternatives to the best route), network routing, and as a building block of optimization and language-processing pipelines. There are two main variants: paths may repeat vertices (*walks*), or they must be *simple* (loopless). For the first variant, Eppstein [12] gave an algorithm that, after one shortest-path tree computation, produces each further path in $O(\log k)$ time (implicitly) and is still the reference today. For simple paths, Yen's algorithm [22] with Lawler's improvement [19] is the classic choice; sidetrack-based algorithms are a recent improvement [17, 23].

This paper is an engineering report. Its starting point is a 486-line C# implementation of Eppstein's idea written by the author in 2010. It computed correct paths on the example of

*This paper is the result of an experiment: the code in all three languages, the tests, the measurements and the text were generated with Claude, Anthropic's AI model, through Claude Code [2], under the author's direction. See section 7.

Eppstein's paper, but it enumerated all paths eagerly in the first query, its priority queue was a sorted list, and it did not terminate on graphs with cycles. In November 2026 the author ran an experiment: could Claude, an AI model used as a coding agent, turn it into a modern library? The project grew in small steps, each one requested by the author: fix the non-termination, port to Linux, add Graphviz output, port to Go, make it fast, add simple paths, add arc flags with real partitions, add contraction hierarchies, port to Rust, and finally write this paper.

Contributions.

- A combination of techniques for k shortest paths in one library (section 3): lazy Eppstein enumeration with persistent leftist heaps, a detour corridor that bounds the tree *and* the enumeration, Yen–Lawler with A* spur searches guided by the exact distances of the shortest-path tree, arc flags with partitions from external partitioners, and a contraction hierarchy with PHAST for the tree. All techniques keep the results exact; every combination is tested against brute force.
- Three implementations with identical interfaces and results in C#, Go and Rust, sharing golden test data, and an identical benchmark (same pseudo-random graphs, same output), which allows a controlled comparison of garbage-collected and non-garbage-collected languages on the same algorithms and data layout (sections 4 and 5).
- A case study of AI-assisted algorithm engineering (section 7): which inputs came from the human (an old algorithm, short requests, expert ideas), what Claude did, and where it failed.

2 Background and related work

Eppstein's algorithm. Let $d(v)$ be the distance from v to t, computed by one run of Dijkstra's algorithm [9] on the reversed graph; the best outgoing edges form the shortest-path tree T. Every edge $e = (u, v) \notin T$ is a *sidetrack* with non-negative *delta* $\delta(e) = w(e) + d(v) - d(u)$, the extra cost of taking e instead of following T. Every s–t walk is uniquely described by its sequence of sidetracks, and its weight is $d(s) + \sum \delta$. Eppstein organizes the sidetracks reachable from each vertex in heaps $H(v)$ that share structure along tree paths (persistence [11]), and explores the resulting *path graph* best-first. Each extracted path creates at most three new candidates. Jiménez and Marzal [15] showed that a lazy variant, which builds heaps only where needed, is faster in practice.

Simple paths. Yen's algorithm [22] derives each new simple path from a previous one by fixing a prefix (the *root*) and computing the best *spur* from the root's last vertex, avoiding the root's vertices and the edges already used by earlier paths with the same root. Lawler [19] observed that only spurs from the deviation vertex onward are needed. Kurz and Mutzel [17] and Al Zoobi et al. [23] combine Eppstein's sidetrack idea with simplicity checks and avoid many spur searches.

Speed-up techniques for shortest paths. Route planning has a rich toolbox [3]. *Arc flags* [18, 20] partition the graph into regions and mark, for every edge and region, whether the edge lies on some shortest path into that region. *Contraction hierarchies* (CH) [13] order the vertices by importance and add shortcuts, so that every shortest path has an equivalent up–down path. *PHAST* [5] computes a full shortest-path tree on a CH with one upward search and a linear sweep; RPHAST [6] restricts it to a target set. *Customizable CH* [8] separate a metric-independent preprocessing from fast re-customization of weights. Alternative routes [1] and k shortest paths with limited overlap [4] address a related problem, where paths must be sufficiently different. We found no work that combines Eppstein's enumeration with CH/PHAST;

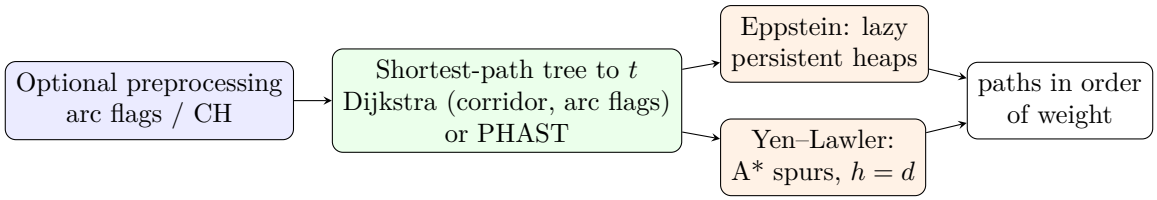


Figure 1: A query. The tree supplies the distances $d(v)$ to the goal, which Eppstein uses for the sidetrack deltas and Yen–Lawler as exact A* lower bounds. The corridor bounds the tree, the enumeration and the spur searches.

since Eppstein needs the distances of all vertices, a point-to-point CH query is not enough, but PHAST is (section 3.6).

Garbage collection. Hertz and Berger [14] quantified that garbage collection can match explicit memory management in speed if it is given several times more memory. Our setting is different: all three implementations avoid allocation in their hot loops, so the comparison is mostly one of compilers, runtime checks and data layout (section 6).

3 Algorithms and their combination

Figure 1 shows how a query proceeds. All techniques keep the results exact: the same paths with the same weights are returned (paths of equal weight may come in a different order).

3.1 Data layout

Vertices and edges are stored in flat arrays (structure of arrays): tail, head, weight and group per edge, and compressed adjacency rows by direction, rebuilt lazily after a change. Per-vertex query data (distance, tree edge, heap root) live in arrays that are reused across queries: a vertex's entry is valid only if its *stamp* equals the current query number, so a query touches only the vertices it reaches and never resets the arrays. Public `Vertex`, `Edge` and `Path` types are small values that refer to these arrays.

3.2 Eppstein with lazy persistent heaps

For every vertex v that the enumeration reaches, $H(v)$ is the persistent merge of $H(\text{next}(v))$ and the sidetracks leaving v; merging copies only the $O(\log n)$ nodes on the right spine of a leftist heap. Heaps are built on demand, iteratively from the goal towards v, so long tree paths cause no deep recursion. A candidate is represented by a pointer to its parent candidate and one heap node, so creating one costs $O(1)$; extracting a candidate creates at most three successors (the two heap children of its last sidetrack and the root of the heap at the head of that sidetrack). Writing out a path costs time proportional to its length, which dominates in practice (section 5).

3.3 Detour corridor

Users rarely want arbitrarily long alternatives. With an absolute or relative detour limit Δ, the backward Dijkstra stops at radius $d(s) + \Delta$: vertices farther away cannot lie on an admissible path. The same bound prunes candidates in the enumeration ($d(s) + \sum \delta \le d(s) + \Delta$) and the A* spur searches. This makes the number of paths finite even with cycles and turns a query between nearby vertices of a large graph into a local computation.

Table 1: Implementations. Lines exclude blank and comment lines.

	library	tests	notable language-specific choices
2010 original (C#)	486	–	object graph, sorted-list priority queue
C# (.NET 8/10)	1825	1212	`readonly struct` views, `PriorityQueue`, FsCheck
Go 1.23+	1851	1733	generic heap, `iter.Seq`, native fuzzing
Rust 2021	1988	352	arenas with `u32` links, borrow-checked iterators

3.4 Simple paths: Yen–Lawler with exact A*

In networks with two-way edges most of Eppstein's walks are the shortest path plus a small loop, not real alternatives. With `SimplePathsOnly`, the library uses Yen's algorithm with Lawler's rule. The found paths form a trie; the edges to block at a root are the children of the root's trie node. Each spur search is an A* search with the tree distances $d(v)$ as potential. Blocking vertices and edges can only make distances longer, so d is a consistent lower bound, and the search explores little more than the spur itself; with a corridor, it stops as soon as no admissible spur can exist. Spurs of a path are generated lazily when the next path is requested, so asking for one path costs no spur search.

3.5 Arc flags

The vertices are partitioned into regions (automatically by a breadth-first order, from a string, or from an external partitioner such as Mt-KaHyPar [21]; the library writes the METIS input format). For every boundary vertex one backward Dijkstra marks the tight edges. A query relaxes only edges flagged for the goal's region. Because Eppstein needs the distance of *every* vertex that can reach the goal, arc flags cannot skip vertices here, only edge relaxations; we measured a gain of about 20% with good partitions, which is why the documentation recommends the corridor first.

3.6 Contraction hierarchy with PHAST

The contraction orders vertices lazily by edge difference plus the number of contracted neighbors, uses witness searches with settled-vertex limits, and keeps the minimum of parallel edges. A query without a corridor then computes all distances with PHAST [5]: an upward search from t over reversed downward edges, followed by one sweep in descending rank over upward edges, with vertices stored in sweep order. Eppstein also needs the tree, not only the distances; we rebuild it from the *original* edges that are tight ($d(u) = w(u, v) + d(v)$), breadth-first from t, which is correct even with zero-weight cycles. With a corridor the bounded Dijkstra visits fewer vertices than PHAST and is used instead.

4 Three implementations

Table 1 summarizes the implementations. They follow the same structure file by file; the interfaces differ only where the languages differ (exceptions vs. error values vs. `Result`, properties vs. methods, `IEnumerable` vs. Go iterators vs. Rust iterators).

Memory management. In C# and Go, heap nodes and candidates are small objects managed by the garbage collector. In Rust they live in two vectors (arenas) per query and refer to each other by `u32` indices; the arenas are cleared at the next query. There is no reference counting and no unsafe code in the library.

Table 2: Eppstein, corner to corner on one-way grids (median times in ms). Tree: repeated query; next: the remaining $k-1$ paths. Fastest per group in bold (within 3%).

	tree			next paths		
	C#	Go	Rust	C#	Go	Rust
200^2, $k = 10^4$	29.2	8.6	**7.3**	79.6	45.5	**35.3**
500^2, $k = 10^4$	54.0	73.4	**52.4**	205	113	**88.6**
1000^2, $k = 10^4$	255	312	260	406	359	314
1000^2, $k = 10^5$	253	331	272	3,419	3,436	3,079

Safety of iteration. All three offer an iterator over the paths of a query. Changing the graph during the iteration is a runtime error in C# (`InvalidOperationException`) and Go (panic). In Rust the iterator borrows the graph mutably, so the same mistake is a compile-time error; reading the graph during the iteration goes through the iterator.

Labels. Vertex labels are case-insensitive. C# and Go keep the caller's string if it is already in upper case; the Rust version stores all label text in one string and indexes it with an open-addressing hash table, which avoids one heap allocation and one map entry per vertex. (A first, straightforward Rust version stored each label twice and needed 200 MB instead of 112 MB for the largest graph.)

Testing. The three implementations read the same golden files (graphs and expected paths, including DIMACS input), and compare against brute-force enumeration on thousands of random small graphs with cycles, zero and negative weights, corridors, arc flags and contraction hierarchies. C# adds property-based tests (FsCheck), Go adds coverage-guided fuzzing of the importers and of the path enumeration. CI enforces coverage gates (C# $\geq 95\%$ lines, Go $\geq 93\%$ statements).

5 Experiments

Setup. All measurements ran on Intel Xeon Processor @ 2.10GHz (4 virtual cores, Linux) with .NET 10.0.114, Go 1.24.7 and rustc 1.97.0 (release builds; Rust with one codegen unit). The benchmark is the same program in the three languages. It builds a $n \times n$ grid (vertex $v = y\,n + x$; edges to the right and lower neighbor, and every tenth vertex also to its left neighbor, which creates cycles); `two-way` grids have every street in both directions. Weights are $1 + (r \bmod 100)$ for r drawn from SplitMix64 [16] with seed 1, implemented identically in all three languages, so all of them build exactly the same graphs. The benchmark prints the weight of the last path, and the measurement script aborts if the languages disagree; they agreed in every scenario. Each scenario ran in a fresh process 5 times per language (once for the CH on 10^6 vertices); we report medians. "Tree" is the second identical query (i. e. without one-time work such as building adjacency rows or JIT compilation), which is dominated by the shortest-path tree; "next" is the time for the remaining $k-1$ paths. Memory is the live heap after the first query: after a full collection in C# and Go, and the allocator's live bytes in Rust.

Eppstein, corner to corner (table 2). Grids with $4 \cdot 10^4$ to 10^6 vertices, paths across the whole graph. All three languages return the same path weights. On the 10^6-vertex grid the tree takes 255 ms (C#), 312 ms (Go) and 260 ms (Rust), and the next 10^4 paths 406, 359 and 314 ms. For 10^5 paths all three need 31–34 µs per path. Most of that time goes into writing out each path of about 2000 edges, not into the enumeration, which creates at most three candidates per path; an iterator over weights only avoids this cost. On the small 200^2 grid C# looks 3–4× slower. As section 6 shows, this is caused by tiered JIT compilation, not by garbage collection.

Table 3: Nearby vertices in the middle of the 10^6-vertex grid, $k = 10^4$ (ms). First: first query (includes one-time work such as JIT compilation and building adjacency rows).

	first query			tree			next paths		
	C#	Go	Rust	C#	Go	Rust	C#	Go	Rust
no corridor	218	115	**98.0**	**65.5**	84.4	69.1	33.3	**10.5**	**10.7**
`MaxDetourRatio` = 0.1	46.8	**21.6**	27.5	3.0	0.80	**0.50**	32.0	**10.7**	**11.0**

Table 4: Building the 10^6-vertex grid ($2.1 \cdot 10^6$ edges), in ms, and the live memory of the graph with one query (built by index).

	C#	Go	Rust
by index	440	530	253
by label strings	1,131	1,057	736
DIMACS text import	560	873	417
live memory (MB)	117	94	112

Corridor (table 3). Two nearby vertices in the middle of the 10^6-vertex grid. Without a corridor, the tree covers the whole graph; with paths limited to 10% above the shortest one, the tree covers only the neighborhood. Without a corridor, the query between two nearby vertices still computes the distances of all 10^6 vertices (65–84 ms). With the corridor, the tree takes 0.5–3.0 ms, more than 100× less in Rust, and 10^4 paths take 11–32 ms. In absolute terms the corridor matters far more than any language or preprocessing choice.

Simple paths (table 5). Two-way grids, where simple paths matter. Each simple path costs many A* spur searches, so the times are much higher than for walks: 0.7–0.9 s for 100 paths and 5.5–7.6 s for 1000 paths on the $4 \cdot 10^4$-vertex grid. Rust is fastest for $k = 100$; for $k = 1000$, C# is 5% faster than Rust, which is within noise. One likely reason is that .NET's built-in `PriorityQueue` is a 4-ary heap, while the Go and Rust ports use binary heaps. Go is slowest here: its trie and duplicate check use maps keyed by strings. With a corridor on the 10^6-vertex graph, 100 simple paths take 35–96 ms.

Contraction hierarchy (table 6). The contraction takes 2.6–3.2 s for the 200^2 two-way grid and 66–74 s for the 10^6-vertex grid, with about three million shortcuts. PHAST then computes the full tree of the 10^6-vertex grid in 76–97 ms instead of 255–312 ms with Dijkstra (2.6–4.0×). Go and Rust produce exactly the same number of shortcuts, because their priority queues break ties identically; C# differs by less than 0.1%. The speed-up is below the order of magnitude reported for road networks [5]: grids are a hard case for contraction hierarchies, and our sweep is single-threaded and does not use SIMD.

The 2010 original (fig. 2). On acyclic grids, where it terminates at all, the original implementation's first query enumerates every path: 156 ms for 10×10 (48,620 paths), 1.8 s for 11×11 and 40 s and 132 MB for 12×12 (705,432 paths). The current version needs 0.1 ms for the first path and less than 3 ms for the next 999, independent of the number of paths in the graph. On graphs with cycles, the original does not terminate.

6 Discussion: garbage collection and language

Table 7 separates the effects of the runtimes. *Garbage collection is not a factor here.* Turning it off completely in Go (`GOGC=off`) or making it rare in C# (a 256 MB generation-0 budget)

Table 5: Simple paths (Yen–Lawler with A*) on two-way grids: time for the paths after the first (ms).

	C#	Go	Rust
200^2, $k = 100$	864	930	666
200^2, $k = 1000$	5,512	7,402	5,773
1000^2 local, ratio 0.1, $k = 100$	95.7	50.9	**34.5**

Table 6: Contraction hierarchy: preprocessing and the PHAST tree (repeated query), in ms, and the number of shortcuts (Rust; C# and Go differ slightly because of tie-breaking in witness searches).

	contraction			tree (PHAST)			
	C#	Go	Rust	C#	Go	Rust	shortcuts
200^2 two-way	3,158	2,947	2,628	5.1	3.2	**2.6**	179,980
1000^2 one-way	74.1 s	70.0 s	65.5 **s**	97.3	78.6	**76.1**	3,051,496

changes no measured time beyond noise. The reason is the data layout, not the collectors: all per-vertex data lives in arrays that are reused across queries, and the only allocations during an enumeration are small, short-lived candidate and heap objects, which generational collectors handle cheaply.

JIT warm-up is a factor. With default settings, .NET first compiles methods quickly without optimization (tier 0) and optimizes them only after many calls or long loops. A benchmark that runs a query twice therefore measures partly unoptimized code. With tiered compilation disabled, the C# tree on the 200^2 grid takes 6.4 ms instead of 27.7 ms (Go 9.0 ms, Rust 6.6 ms), and Yen's tree 8.6 ms instead of 38.1 ms. For long-running services this does not matter; for command-line tools and short benchmarks it dominates.

What remains is compilers and libraries. Rust is fastest in most scenarios. Its largest advantages are in building the graph (253 ms by index versus 440 and 530 ms) and in path enumeration (10–30%), where arenas with 32-bit links replace small objects. In the hot loops of Dijkstra and A*, all three languages do the same array accesses with bounds checks, and their times are within about 25% when C# runs optimized code. Library choices also matter at this level: the 4-ary heap of .NET versus our binary heaps, and Go maps keyed by strings versus a C# hash set or a Rust hash set.

Memory. The live heap of the 10^6-vertex graph with one query is similar: 117 MB (C#), 94 MB (Go) and 112 MB (Rust). Go and C# share the caller's label strings, which are therefore not counted. The live heap understates what a garbage-collected process needs, because collectors run with headroom [14]; we did not measure peak resident memory.

Engineering effort. The Rust port was the third implementation and was generated by Claude from the Go version. Two Rust-specific properties stood out. The borrow checker turned the "graph changed during iteration" error of C# and Go into a compile-time error, at the price of a slightly less convenient iterator. And the most natural Rust structure for persistent heaps, `Rc` pointers, was replaced by index-based arenas, which are both faster and simpler to reason about than reference counting.

7 AI-assisted development

7.1 Process

The whole project is an experiment with Claude [2]. The author worked with it as a coding agent in a cloud development environment with access to the repository. Claude read and wrote the

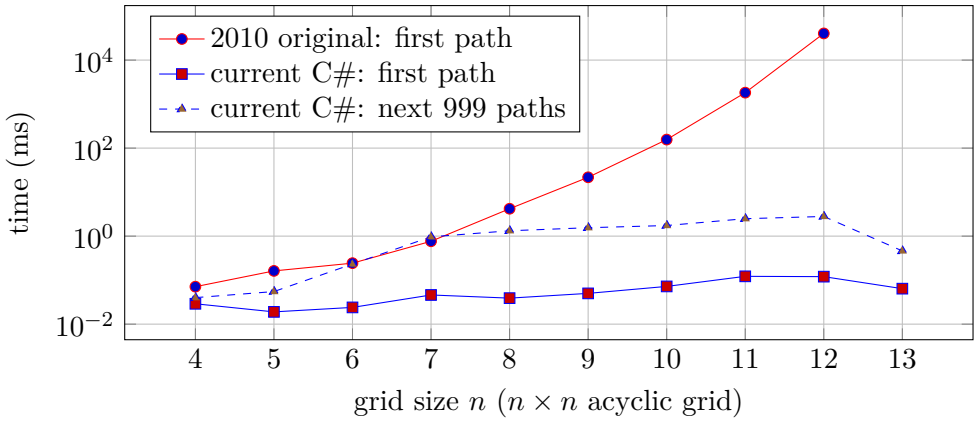


Figure 2: The 2010 implementation enumerates all $\binom{2(n-1)}{n-1}$ paths in the first query; the current one computes one tree and then each path on demand.

Table 7: Runtime settings (median ms). Turning garbage collection off (Go) or making it rare (C#) has no measurable effect; disabling tiered compilation makes short C# runs 2–4× faster.

	Eppstein 200^2, $k = 10^4$		Eppstein 1000^2, $k = 10^5$		Yen 200^2, $k = 10^3$	
	tree	next	tree	next	tree	next
C# default	27.7	76.2	255	3,441	38.1	5,485
C# gen0 budget 256 MB	28.5	78.9	257	3,489	38.4	5,428
C# without tiered JIT	6.4	42.9	248	3,332	8.6	5,331
Go default	9.0	43.9	324	3,468	10.9	7,581
Go `GOGC=off`	8.0	42.7	318	3,451	12.4	7,646
Rust	6.6	34.8	259	3,092	9.4	5,869

code, ran builds, tests, benchmarks and a fuzzer, searched the literature, opened pull requests and reacted to CI results; the author decided what to do next, chose among the options it proposed, and merged. All work happened in two sessions over two days (8–9 November 2026); 11 pull requests were merged before this paper. The author's requests were short and informal, usually one sentence, and never a specification.

7.2 What the human contributed

Three kinds of input shaped the result. *The old algorithm* fixed the interface (labels, groups, `FindShortestPath`/`FindNextShortestPath`) and the domain; much of the later work kept that interface or broke it only when the author allowed it. *Open questions* (what would improve the project most, how to make it much faster, what is missing for a high-quality library) delegated the search for improvements to Claude, while the author kept the decisions by choosing among the proposed items. *Expert ideas* (arc flags with a real partitioner, contraction hierarchies, the idea for a paper) came from a colleague's review and redirected the work towards algorithm engineering. Explicit constraints, such as excluding multithreading or allowing breaking interface changes, were the most effective requests.

7.3 What Claude did, and where it failed

Claude wrote all code, tests, documentation and benchmarks of version 2, ran every measurement in this paper, and proposed most of the ideas that were implemented. It also argued *against* ideas when its measurements did not support them: it first advised against contraction hierarchies, because the measured bottleneck of long queries was the number of spur searches, and after

being asked to implement them it reported that the gain on grids (3–10× for the tree) is below the order of magnitude reported for road networks. The failures we observed were caught by tests, not by review:

- Hand-computed expected values in new tests were wrong several times (e. g. the weights of simple paths in a small two-way graph, and the edges of Eppstein's example typed from memory in the Rust tests); in each case brute force or the shared golden data showed that the library was right and the expectation wrong.
- Fuzzing found that the Go DIMACS importer treated some Unicode white space differently from line splitting, and that Claude's own test harness had an $O(n^2)$ sort.
- A scripted text edit broke a Go struct tag; the build caught it.
- The first Rust port stored labels twice; the cross-language memory comparison exposed it.
- The environment could not download the DIMACS road networks (HTTP-only server behind an HTTPS-only proxy), so all experiments use synthetic grids (section 8).

7.4 Observations

The decisive factor for reliability was the test infrastructure that Claude built early: golden data shared by all implementations, brute-force oracles and fuzzing. It made large refactorings (object graph to arrays; three languages) cheap to verify and let the author accept changes without reading every line. Claude was most useful as an algorithm engineer when asked open questions and given freedom, and most precise when given constraints. It never needed a formal specification; it needed a way to check itself.

8 Limitations

All experiments use synthetic grids, which are a hard case for contraction hierarchies and favorable for corridors; real road networks (DIMACS [7], OpenStreetMap) are the obvious next step. The machine is a shared cloud VM; we report medians of fresh processes, but differences below about 10–20% should be read as noise. The memory numbers are measured differently in each language (section 5). The case study has a sample size of one author and one AI model, and this paper itself was generated with Claude, so its account of the process is not independent.

9 Conclusion

A small 2010 prototype became a library that combines Eppstein's enumeration, a detour corridor, Yen–Lawler with exact A*, arc flags and PHAST on a contraction hierarchy, in three languages with identical results. Algorithmic choices changed the running time by orders of magnitude; the choice between a garbage-collected and a non-garbage-collected language changed it by tens of percent, once the data layout avoided allocation in the hot loops. Promising next steps are the sidetrack-based simple-path algorithms [17, 23] combined with (R)PHAST or CCH for the repeated tree computations, and experiments on real road networks. The code, the data and the scripts that produced every number in this paper are available in the repository [10].

References

[1] Ittai Abraham, Daniel Delling, Andrew V. Goldberg, and Renato F. Werneck. Alternative routes in road networks. *ACM Journal of Experimental Algorithmics*, 18, 2013.

[2] Anthropic. Claude code: an agentic coding tool. https://claude.com/claude-code. Accessed November 2026.

[3] Hannah Bast, Daniel Delling, Andrew Goldberg, Matthias Müller-Hannemann, Thomas Pajor, Peter Sanders, Dorothea Wagner, and Renato F. Werneck. Route planning in transportation networks. In *Algorithm Engineering: Selected Results and Surveys*, volume 9220 of *LNCS*, pages 19–80. Springer, 2016.

[4] Theodoros Chondrogiannis, Panagiotis Bouros, Johann Gamper, Ulf Leser, and David B. Blumenthal. Finding k-shortest paths with limited overlap. *The VLDB Journal*, 29:1023–1047, 2020.

[5] Daniel Delling, Andrew V. Goldberg, Andreas Nowatzyk, and Renato F. Werneck. PHAST: Hardware-accelerated shortest path trees. *Journal of Parallel and Distributed Computing*, 73(7):940–952, 2013.

[6] Daniel Delling, Andrew V. Goldberg, and Renato F. Werneck. Faster batched shortest paths in road networks. In *11th Workshop on Algorithmic Approaches for Transportation Modelling, Optimization, and Systems (ATMOS 2011)*, volume 20 of *OASIcs*, pages 52–63, 2011.

[7] Camil Demetrescu, Andrew V. Goldberg, and David S. Johnson, editors. *The Shortest Path Problem: Ninth DIMACS Implementation Challenge*, volume 74 of *DIMACS Series in Discrete Mathematics and Theoretical Computer Science*. American Mathematical Society, 2009.

[8] Julian Dibbelt, Ben Strasser, and Dorothea Wagner. Customizable contraction hierarchies. *ACM Journal of Experimental Algorithmics*, 21, 2016.

[9] Edsger W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1:269–271, 1959.

[10] Nick Dinges. k-shortest-path: k shortest paths in C#, Go and Rust. https://github.com/NickDinges/k-shortest-path, 2026.

[11] James R. Driscoll, Neil Sarnak, Daniel D. Sleator, and Robert E. Tarjan. Making data structures persistent. *Journal of Computer and System Sciences*, 38(1):86–124, 1989.

[12] David Eppstein. Finding the k shortest paths. *SIAM Journal on Computing*, 28(2):652–673, 1998.

[13] Robert Geisberger, Peter Sanders, Dominik Schultes, and Daniel Delling. Contraction hierarchies: Faster and simpler hierarchical routing in road networks. In *Experimental Algorithms (WEA 2008)*, volume 5038 of *LNCS*, pages 319–333. Springer, 2008.

[14] Matthew Hertz and Emery D. Berger. Quantifying the performance of garbage collection vs. explicit memory management. In *Proceedings of OOPSLA 2005*, pages 313–326. ACM, 2005.

[15] Víctor M. Jiménez and Andrés Marzal. Computing the K shortest paths: A new algorithm and an experimental comparison. In *Algorithm Engineering (WAE 1999)*, volume 1668 of *LNCS*, pages 15–29. Springer, 1999.

[16] Guy L. Steele Jr., Doug Lea, and Christine H. Flood. Fast splittable pseudorandom number generators. In *Proceedings of OOPSLA 2014*, pages 453–472. ACM, 2014.

[17] Denis Kurz and Petra Mutzel. A sidetrack-based algorithm for finding the k shortest simple paths in a directed graph. In *27th International Symposium on Algorithms and Computation (ISAAC 2016)*, volume 64 of *LIPIcs*, pages 49:1–49:13, 2016.

[18] Ulrich Lauther. An extremely fast, exact algorithm for finding shortest paths in static networks with geographical background. In *Geoinformation und Mobilität*, volume 22 of *IfGIprints*, pages 219–230. 2004.

[19] Eugene L. Lawler. A procedure for computing the K best solutions to discrete optimization problems and its application to the shortest path problem. *Management Science*, 18(7):401–405, 1972.

[20] Rolf H. Möhring, Heiko Schilling, Birk Schütz, Dorothea Wagner, and Thomas Willhalm. Partitioning graphs to speedup Dijkstra's algorithm. *ACM Journal of Experimental Algorithmics*, 11, 2006.

[21] Mt-KaHyPar developers. Mt-KaHyPar: Multi-threaded Karlsruhe graph and hypergraph partitioner. https://github.com/kahypar/mt-kahypar. Accessed November 2026.

[22] Jin Y. Yen. Finding the k shortest loopless paths in a network. *Management Science*, 17(11):712–716, 1971.

[23] Ali Al Zoobi, David Coudert, and Nicolas Nisse. Finding the k shortest simple paths: Time and space trade-offs. *ACM Journal of Experimental Algorithmics*, 28, 2023.